МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Київський національний університет будівництва і архітектури
Кафедра кібербезпеки та комп`ютерної інженерії

ЗВІТ

з лабораторної роботи № 7

Обробка відеопотоку та базовий комп’ютерний зір (OpenCV + ROS 2)

Виконав: студент групи БІКСм-26
Тарасов Максим

Перевірив: Маринський К.В

Київ – 2026

1. Опис Vision Node

Метою роботи було отримати відеопотік у ROS 2, виконати базову обробку зображення за допомогою OpenCV та вивести результат на екран. Роботу виконано у віртуальній машині з Ubuntu та ROS 2 дистрибутива Lyrical, у робочому просторі ros2_ws із попередніх лабораторних робіт. Реальної камери на віртуальній машині немає, тому відеопотік симулюється окремою нодою, яка публікує синтетичні кадри.

Система складається з двох нод пакета vision_node, що обмінюються даними через топік /image_raw (тип sensor_msgs/msg/Image):

Таблиця 1 – Ноди та топіки системи комп’ютерного зору

Нода	**Роль**	**Топік**	**Тип повідомлення**
test_image_publisher	імітація камери: публікує кадр 640×480 кожні 0,1 с (10 кадрів/с)	/image_raw (публікація)	sensor_msgs/msg/ Image
vision_node	отримує кадри, обробляє їх через OpenCV і показує результат	/image_raw (підписка)	sensor_msgs/msg/ Image

Спочатку встановлено необхідні пакети: ros-lyrical-cv-bridge (міст між ROS-повідомленнями та зображеннями OpenCV), ros-lyrical-image-tools, python3-opencv та v4l-utils (утиліти для роботи з камерами). Пакет cv_bridge виявився вже встановленим в останній версії (4.1.0) (рисунок 1).

```
sudo apt update
sudo apt install ros-lyrical-cv-bridge ros-lyrical-image-tools \
    python3-opencv v4l-utils
```

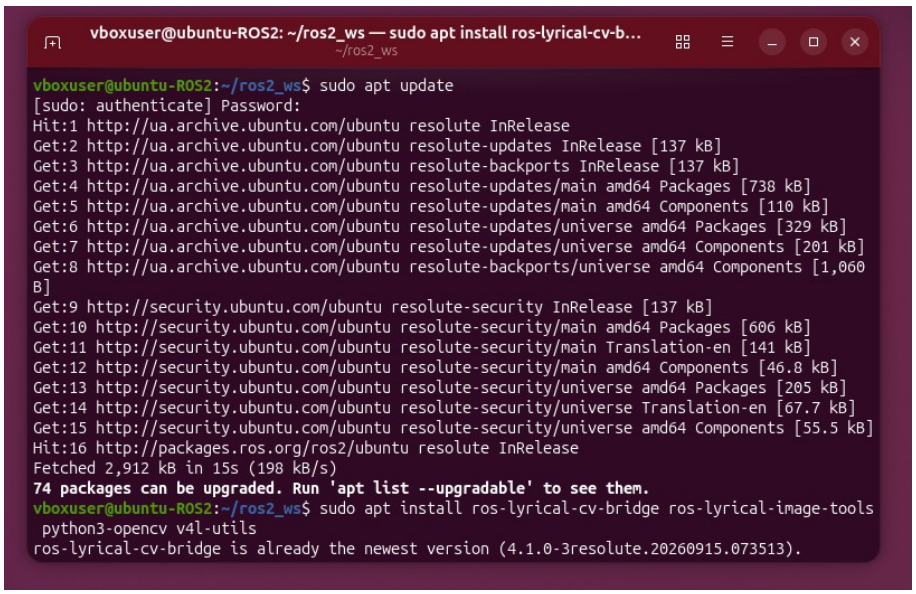


Рисунок 1 – Оновлення індексів пакетів та встановлення cv_bridge і OpenCV

Далі підключено середовище ROS 2 і перевірено доступні топіки: у списку є лише службові /parameter_events та /rosout, тобто камера або її симуляція ще не запущені. Топіка камери /camera/image поки немає, тому для тестування створюється власне джерело зображень. Потім у каталозі ~/ros2_ws/src створено пакет vision_node типу ament_python із залежностями rclpy, sensor_msgs та cv_bridge (рисунок 2):

```
source /opt/ros/lyrical/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 topic list
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python vision_node \
    --dependencies rclpy sensor_msgs cv_bridge
```

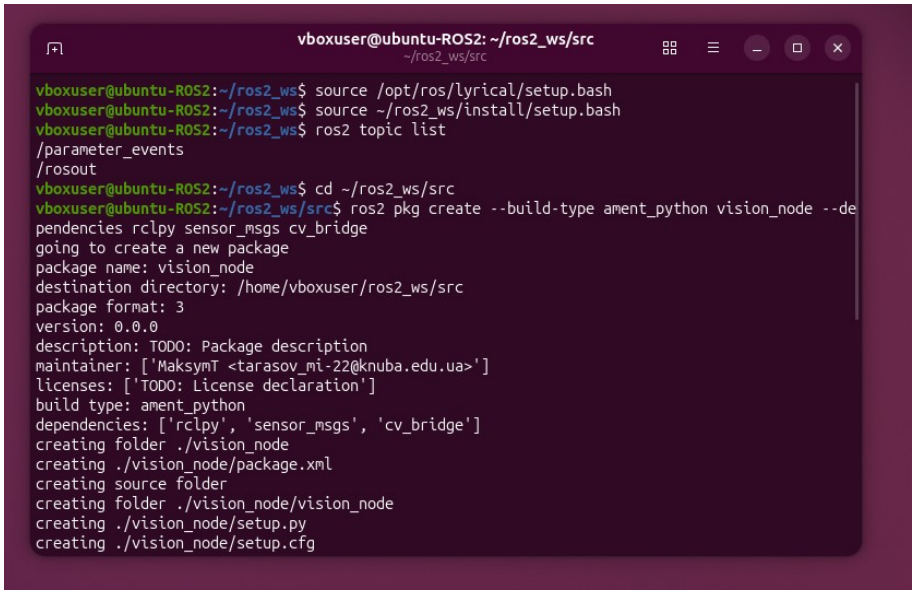


Рисунок 2 – Перевірка топіків та створення пакета vision_node

Для запуску нод командою ros2 run у файлі setup.py до розділу entry_points спочатку додано точку входу vision_node (рисунок 3), після чого пакет зібрано командою colcon build --packages-select vision_node. Збірка завершилась успішно за 2.31 с (загальний час 2.84 с) (рисунок 4).

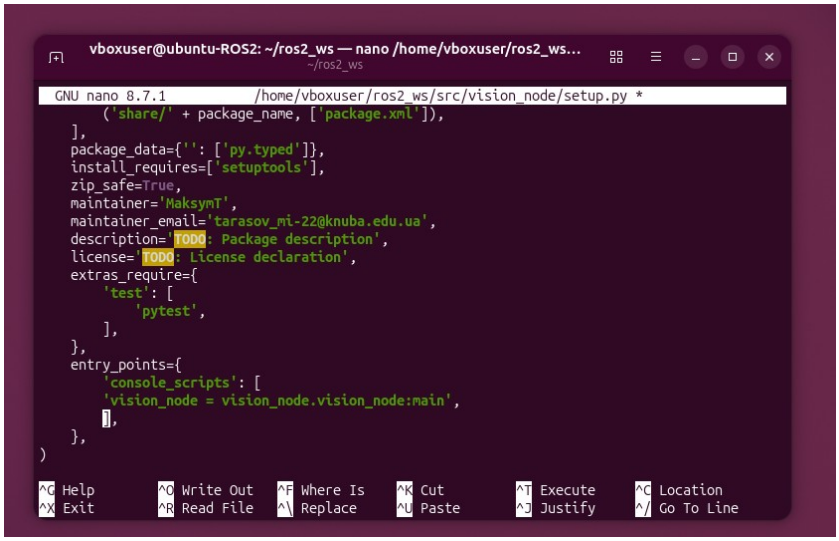


Рисунок 3 – Точка входу vision_node у файлі setup.py

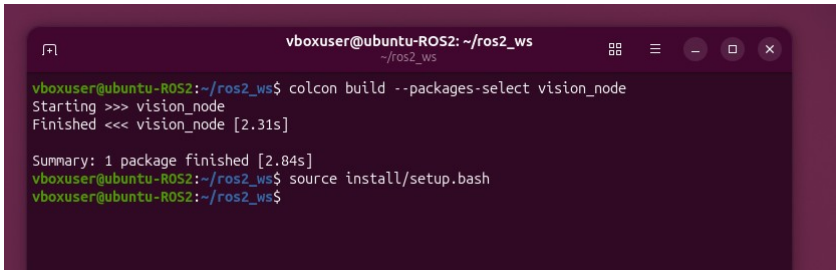


Рисунок 4 – Збірка пакета vision_node

Пізніше до setup.py додано другу точку входу test_image_publisher для тестового публікатора зображень (рисунок 5); оскільки entry_points змінено, пакет потрібно перезібрати командою colcon build, щоб нода стала доступною для ros2 run.

```
entry_points={
    'console_scripts': [
        'vision_node = vision_node.vision_node:main',
        'test_image_publisher = vision_node.test_image_publisher:main',
    ],
},
```

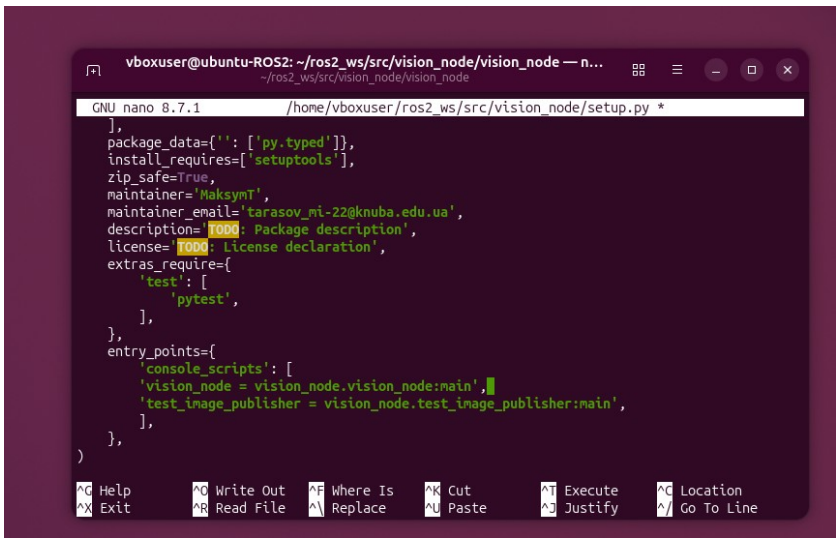


Рисунок 5 – Додавання точки входу test_image_publisher

2. Фрагменти коду

2.1. Тестовий публікатор зображень

Нода test_image_publisher замінює камеру. За таймером з періодом 0,1 с вона створює чорний кадр 640×480 пікселів, малює на ньому зелений квадрат 100×100, що зміщується праворуч на 5 пікселів з кожним кадром (координата x = (counter · 5) mod 500), та підпис з номером кадру. Кадр перетворюється на повідомлення Image за допомогою cv_bridge і публікується в /image_raw:

```
self.publisher_ = self.create_publisher(Image, '/image_raw', 10)
self.bridge = CvBridge()
self.timer = self.create_timer(0.1, self.timer_callback)

def timer_callback(self):
    img = np.zeros((480, 640, 3), dtype=np.uint8)
    x = int((self.counter * 5) % 500)
    cv2.rectangle(img, (x, 200), (x+100, 300), (0, 255, 0), -1)
    cv2.putText(img, f'Frame: {self.counter}', (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)

    msg = self.bridge.cv2_to_imgmsg(img, encoding='bgr8')
    self.publisher_.publish(msg)
    self.counter += 1
```

2.2. Підписка на топік

Vision Node підписується на топік /image_raw з типом повідомлення sensor_msgs/msg/Image. Кожен отриманий кадр передається у функцію зворотного виклику image_callback. Назву топіка вказано в коді; якщо камера публікує в іншому топіку (наприклад, /camera/image), достатньо змінити цей рядок:

```
self.subscription = self.create_subscription(
    Image,
    '/image_raw',          # зміни, якщо у тебе інший топік
    self.image_callback,
    10
)
```

2.3. Перетворення за допомогою cv_bridge

Повідомлення sensor_msgs/Image містить масив байтів та метадані (розмір, кодування), тому для обробки його перетворюють у масив NumPy формату BGR за допомогою CvBridge. Перетворення виконано в блоці try/except: у разі помилки нода виводить повідомлення рівня ERROR і пропускає кадр, не завершуючи роботу:

```
self.bridge = CvBridge()

try:
    # Конвертація ROS Image → OpenCV
    cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
except Exception as e:
    self.get_logger().error(f'Помилка cv_bridge: {e}')
    return
```

2.4. Обробка зображення

Обробка складається з перетворення в градації сірого, розмиття за Гаусом, порогової обробки та (додатково) виявлення країв методом Канні:

```
gray = cv2.cvtColor(cv_image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
_, thresh = cv2.threshold(blurred, 127, 255, cv2.THRESH_BINARY)
edges = cv2.Canny(blurred, 50, 150)

cv2.imshow('Original', cv_image)
cv2.imshow('Grayscale + Blur', blurred)
cv2.imshow('Threshold', thresh)
cv2.imshow('Edges (Canny)', edges)
cv2.waitKey(1)
```

Функція cv2.waitKey(1) потрібна для оновлення вікон OpenCV. У функції main перехоплюється KeyboardInterrupt, а в блоці finally нода знищується, ROS 2 вимикається та закриваються всі вікна (cv2.destroyAllWindows()). Повний код обох нод додається до звіту у файлах vision_node.py та test_image_publisher.py.

3. Опис алгоритму обробки

Кожен кадр, що надходить у /image_raw, проходить такий конвеєр обробки (таблиця 2): кольорове зображення перетворюється в півтонове, згладжується, після чого з нього незалежно отримують бінарне зображення та карту країв.

Таблиця 2 – Етапи обробки зображення

Крок	**Функція OpenCV**	**Параметри**	**Призначення**
1. Градації сірого	cv2.cvtColor	COLOR_BGR2GRAY	з трьох каналів BGR отримується один канал яскравості Y = 0.299·R + 0.587·G + 0.114·B
2. Розмиття Гауса	cv2.GaussianBlur	ядро 5×5, σ = 0	згладжує шум і дрібні перепади яскравості, готує зображення для порога та країв
3. Порогова обробка	cv2.threshold	поріг 127, макс. 255, THRESH_BINARY	пікселі яскравіші за 127 стають білими (255), решта – чорними (0)
4. Виявлення країв	cv2.Canny	пороги 50 і 150	знаходить контури об’єктів за різкими змінами яскравості

На прикладі тестового кадру результат можна передбачити. Яскраво-зелений квадрат (0, 255, 0) після перетворення в півтони має яскравість Y = 0.587 · 255 ≈ 150, тобто на сірому зображенні виглядає як середньо-сірий. Оскільки 150 > 127, під час порогової обробки квадрат стає повністю білим, а чорний фон (яскравість 0) залишається чорним. Білий текст «Frame: N» має яскравість 255 і в усіх зображеннях видно чітко. Метод Канні виділяє лише

межі: на карті країв видно тонкий контур квадрата та контур літер без заповнення.

4. Скриншоти та результати виконання

Для перевірки запущено два процеси: ros2 run vision_node test_image_publisher (імітація камери) та ros2 run vision_node vision_node (обробка). Після запуску відкрились чотири вікна OpenCV: «Original», «Grayscale + Blur», «Threshold» та «Edges (Canny)» (рисунок 6). Лічильник кадрів у момент знімка дорівнював 874, що за частоти 10 кадрів/с відповідає приблизно 87 с після запуску публікатора.

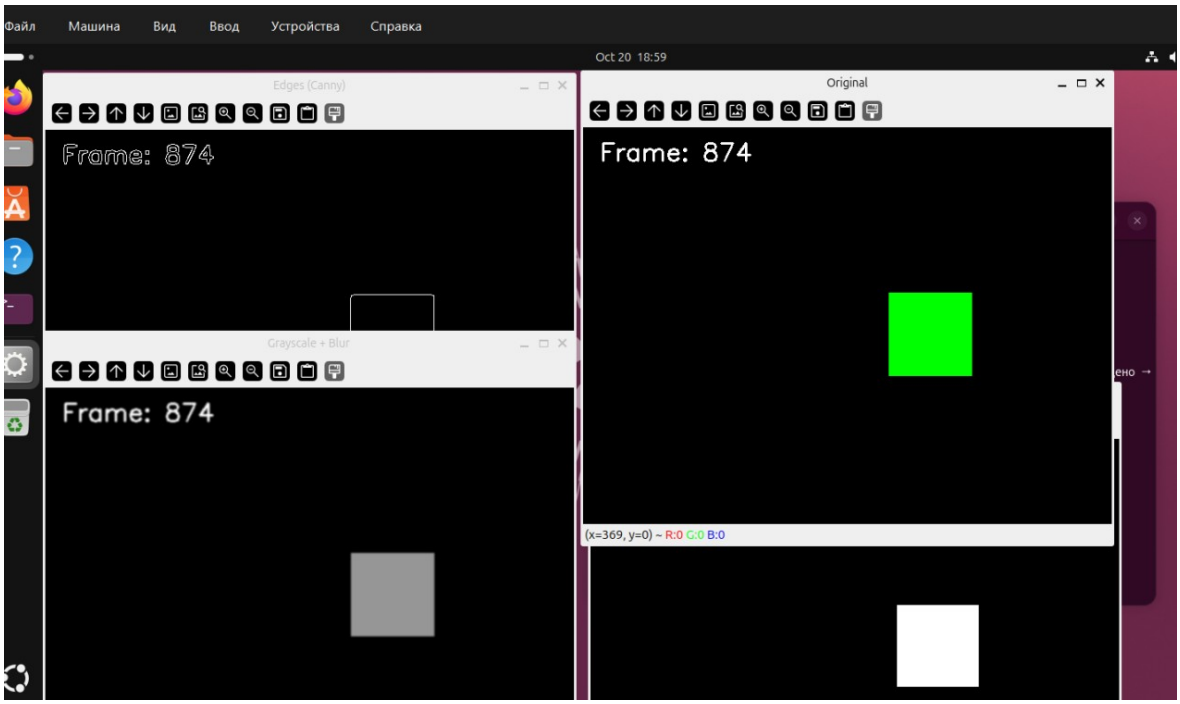
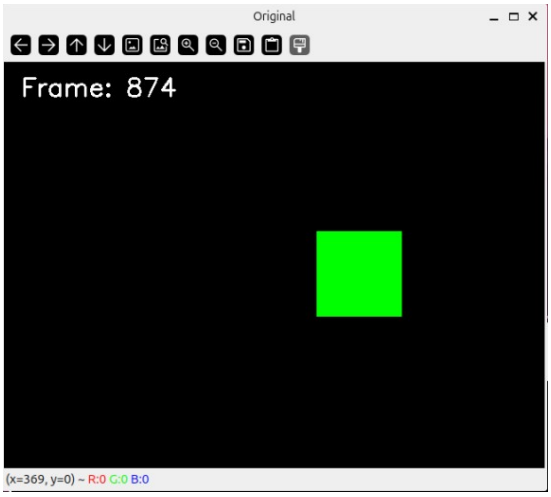
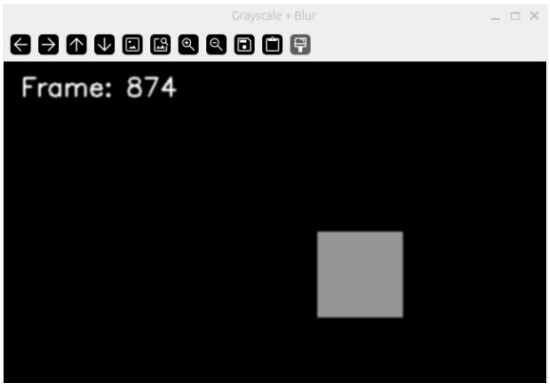


Рисунок 6 – Вікна Vision Node: оригінал, градації сірого з розмиттям, пороговий та Canny

Окремі вікна з рисунка 6 наведено на рисунку 7 для порівняння оригінального та обробленого зображень:



а) оригінал (Original)



б) градації сірого + розмиття



в) краї (Canny)



г) порогова обробка (фрагмент)

Рисунок 7 – Оригінальне та оброблені зображення (кадр 874)

Отримані результати відповідають очікуванням:

- оригінал: зелений квадрат на чорному фоні з написом «Frame: 874»; положення квадрата x = (874 · 5) mod 500 = 370 пікселів збігається з розрахунковим;
- градації сірого з розмиттям: квадрат став середньо-сірим (яскравість ≈ 150), краї трохи згладжені;
- порогова обробка: квадрат повністю білий, фон чорний – зображення бінарне;
- краї Canny: видно лише контур квадрата та літер напису.

Таким чином, відеопотік надходить (лічильник кадрів змінюється), обробка працює, а результат відображається в реальному часі.

5. Висновки

У ході лабораторної роботи створено пакет vision_node на Python (ament_python) із залежностями rclpy, sensor_msgs та cv_bridge і розроблено дві ноди: тестовий публікатор test_image_publisher, що імітує камеру й публікує кадри 640×480 у топік /image_raw із частотою 10 Гц, та Vision Node, яка підписується на цей топік, перетворює повідомлення Image у формат OpenCV за допомогою cv_bridge та обробляє зображення.

Реалізовано послідовність обробки: перетворення в градації сірого, розмиття за Гаусом з ядром 5×5, порогова обробка з порогом 127 та виявлення країв методом Канні; результат виводиться у чотири вікна cv2.imshow. Експеримент підтвердив коректну роботу: на кадрі 874 квадрат на оригіналі, півтоновому, пороговому зображеннях і карті країв має очікуваний вигляд. Помилки перетворення обробляються з виведенням повідомлення рівня ERROR, тому окремий поганий кадр не зупиняє ноду.

Отриманий підхід можна застосувати до реальної камери: достатньо змінити назву топіка в підписці (наприклад, на /camera/image). Подальший розвиток – виявлення руху (різниця сусідніх кадрів), пошук кольорових об’єктів і передача результатів розпізнавання в систему керування рухом робота.